Universität Zürich UZH

GEO877

# Spatial Algorithm

## 05. Points to polygon (+ buffer)

Inhye Kong

# Feedback/solution for your practical 4

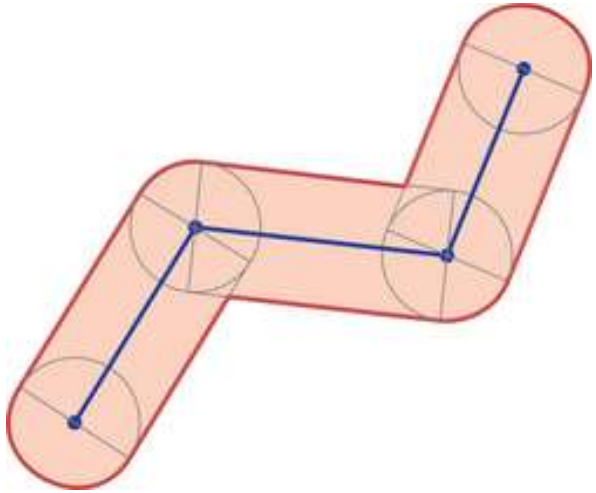
- Practical 4 was about testing whether point(s) is inside a polygon

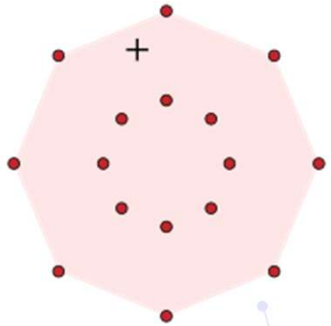
- Let’s take a look at Solution together

# Buffer *

- Buffer is to add certain distance in all sides of given point, segment, and polygon

- Basic algorithm on Wikipedia says
  1. Create a circle around all points
  2. For a set of segments, create distanced segments to make rectangles
  3. Merge the circles and rectangles
     - One approach is to generate some points around a circle, and make a buffer around it (inspired by Minkowski sums)






https://en.wikipedia.org/wiki/Buffer_analysis
https://en.wikipedia.org/wiki/Minkowski_addition

# `PointGroup` class

- Thematically associated group of points (ex. Trees of Canton X and Canton Y)

- Property:
  - A collection of Points, without specific spatial ordering

- Behaviour:
  - Minimum or maximum point of a group
  - **Area to enclose the point group**
    - **Bounding box**
    - **Convex hull**
  - Size (area) and centroid of point group

# PointGroup to Polygon

- One measure you already learnt:
- **Bounding box** is one proximate way to delineate the ‘boundary’ of a PointGroup. But this is incomplete, often applied as ‘filtering’ measure.
- How did you make a bounding box?

- Then how do we make the real polygon out of a PointGroup?

# PointGroup to Polygon

A task of 'boundary generation'.
When do we need this?

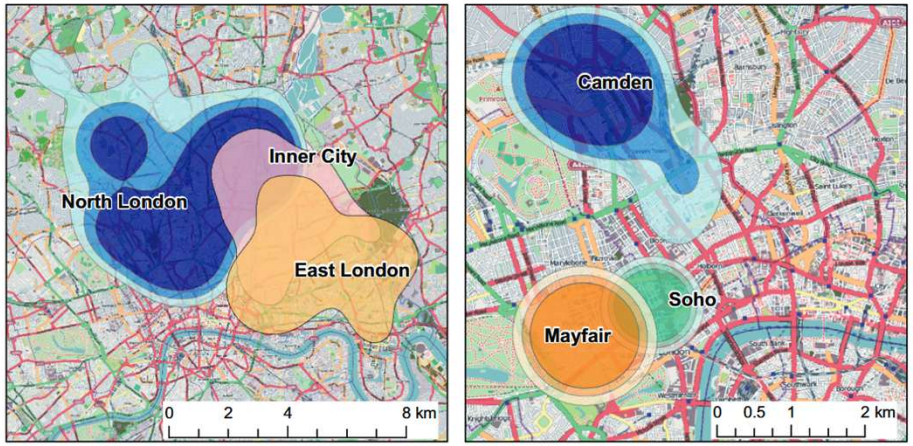
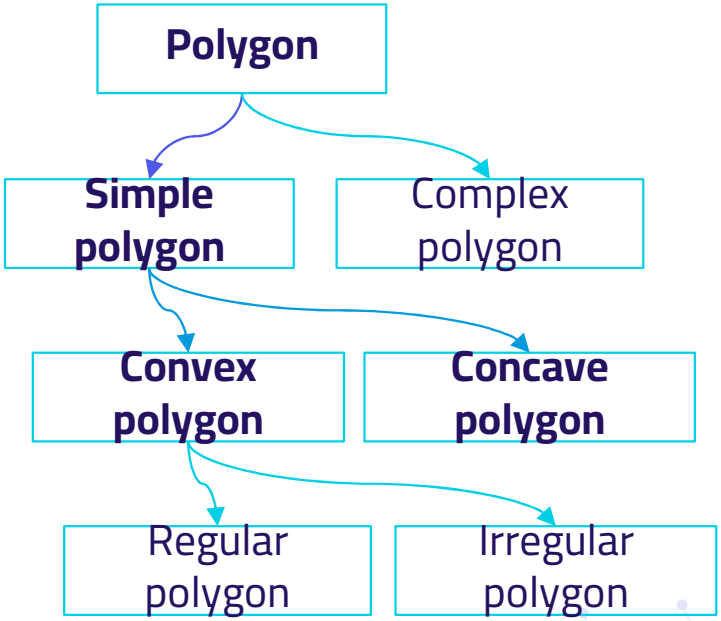



Figure 8: Generic ("northlondon," "innercity," and "eastlondon") and specific ("mayfair," "soho," and "camden") place names in London. For "innercity" and "eastlondon" only 50% contours of the volume surface are shown, for all other regions 50, 60, and 70%. London background mapping openstreetmap.org licensed under a Creative Commons Attribution-Share Alike License.

Also, what kind of polygon do we want?




Hollenstein & Purves. 2010. Exploring place through user-generated content: Using Flickr tags to describe city cores

# PointGroup to Polygon

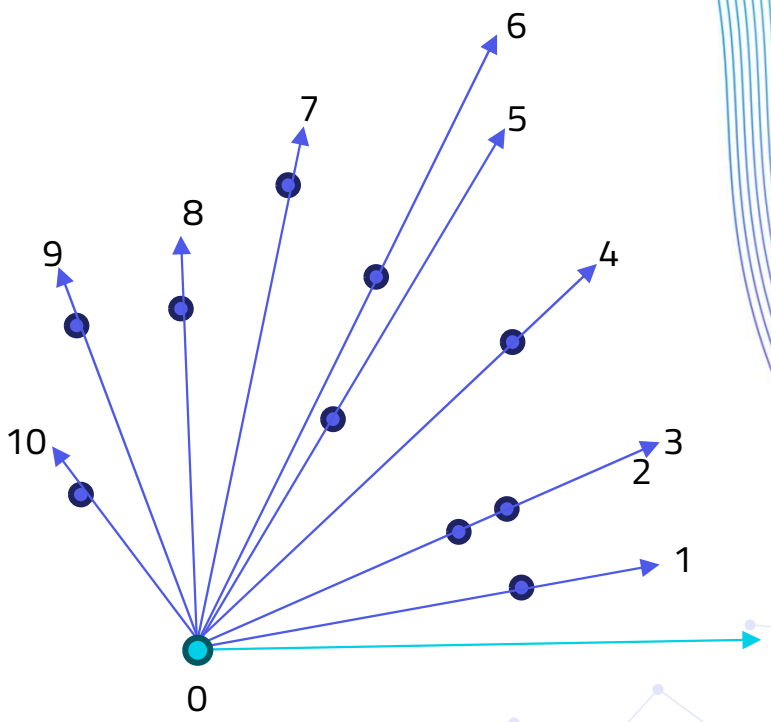
I propose an angle-based approach, called the **Graham Scan** to find a **convex hull** – the smallest convex polygon

1. A way of finding the minimum point in a PointGroup
2. A way of sorting Points by angle, i.e. giving an order to Points (assuming we work with unsorted PointGroup)
3. An implementation of Graham Scan Algorithm to decide whether Points lie on the hull or not, using our turn test

When you look at step 1-2, can you think how/why this approach works?

# Graham Scan: Step 1, 2a

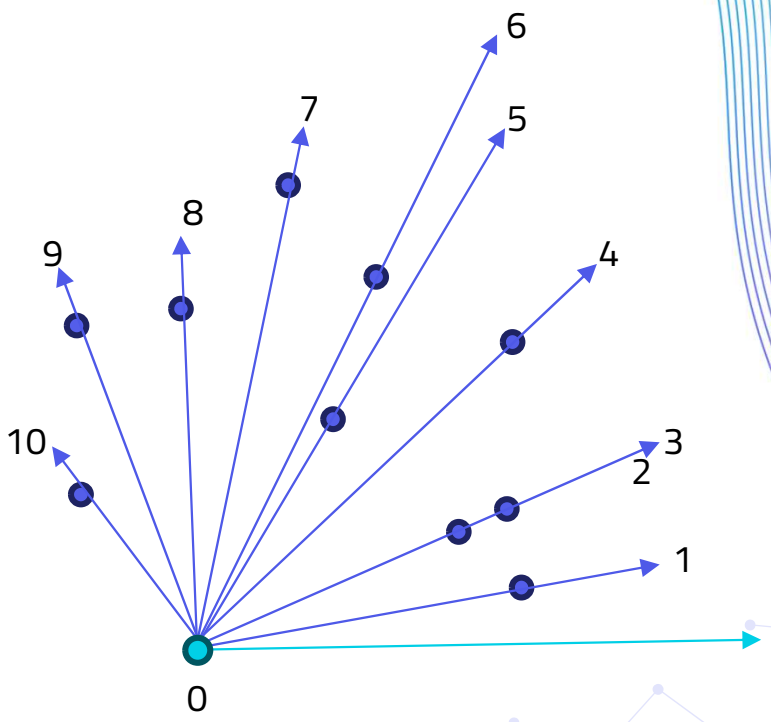
1. Find the minimum point of y (and x if ties) in a PointGroup and set as an anchor
2a. Sort the remaining Points by angle
    - Connect the points with the anchor point and measure the angle
    - The angles will always between 0 and $\pi$. Why?
    - How can we measure the angle?
    - We don't need to "measure" the angle – just a **cosine** function of two points, which will always between 1 and -1.



# Graham Scan : Step 2b

2b. When sorting the remaining Points by angle

- We need to handle a special case: collinear point(s)
- We can remove the one(s) that has shorter distance(s) out of collinear points



# Graham Scan : Step 3

3. We decide whether Points lie on the **convex hull** or not

- We can use two characteristics of a convex polygon:
  - a segment connecting any two points lies inside the hull (including the boundary)
  - all **internal** angles $< \pi$ (180°)

- We apply a turn test following the sorted points, to test internal angles
  - while Point q is not the origin (from the sequence a, p, q)
    - If a, p, q make a left turn, we advance a, p, q by one
    - If a, p, q make a right turn, p is inside the hull, and we set q to be p
  - We build a new data structure for collection, called a **stack** (i.e. List in python)

# Pseudocode for Step 3

- First two points in the sorted set are always on the hull, unless there are collinear points (not tested here)
- We start iterating with 3rd point, and make a triplet to apply a turn test
  - If the turn test is positive, it means a left turn; then we add p to the hull, and increment a
  - If the turn test is zero or negative, it means a collinear or a right turn; then we can tell p is in the polygon and discard it

```
points = points list sorted (Step 2)
n = number of points in the list

hull = []
hull.append(points[0])
hull.append(points[1])
a_position = 1

For (i = 2, i < n-1, i++):
    a = points[a_position]
    p = points[i]
    q = points[i+1]

    if turn test is left:
        hull.append(p)
        a_position = i

hull.append(points[n])
hull.append(points[0])
```

# Practical exercise 5:

- Before we move on the codes, let's look at the supplementary material that visualizes these steps

- I prepared three different sample dataset
- Let's do it together this time; you will try to understand Step1, 2, and 3 of Graham Scan method, written in descriptive code
- Your task is to modify this as an object-oriented method and integrate in our full code stacks we have been accumulating

How others explain this? https://www.geeksforgeeks.org/dsa/convex-hull-using-graham-scan/
https://ninamartin.io/algorithms/2021/04/11/graham-scan.html

# Next time:
# Last lecture from my side

- I will handle spatial indexing for the last lecture (**April 23**)
- In the following week (**April 30**), you will make a brief progress presentation (10 minutes, and it will not be graded), to share your defined research question, your 3 spatial algorithms, how your group works together, and challenges you are facing at the moment